SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team Shrooms

Hackathon Lead

danielzfchan@gmail.com

Clinical Diagnostics Triage & Workflow Copilot

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Key Elements Covered 4

1.3 Solution Overview 4

2. Background 5

2.1 Problem Statement 5

2.2 Previous State 5

2.3 Changes in Major Architectural Components 6

3. Target Stakeholders & Roles 7

4. System Architecture & Design 8

4.1 High-Level Architecture Overview 8

4.2 Clinical Reasoning and Local Processing Flow 9

4.2.1 Prompt Construction, Context Management, and Summarization 9

4.2.2 Multi-File Processing and Modality Routing 9

4.2.3 Response Parsing, Safety Checks, and Mock FHIR R4 Output 10

4.3 Sequence Diagram 11

4.4 Technology Stack 12

4.5 Data Flow Diagram (DFD) 13

4.6 Data Schema - Mock FHIR R4 Output Structure 15
5. Diagnostic Models 17
5.1 ECG Signal Classification 17
5.2 Medical Imaging Analysis 17
5.3 Clinical Note NLP Processing 18
6. Functional Requirements & Scope 19
6.1 Core Functional Capabilities 19
6.2 Non-Functional Requirements 20
6.3 Out of Scope / Future Enhancements 21
7. Monitoring, Evaluation, Assumptions & Dependencies 22
7.1 Technical Evaluation Strategy 22
7.1.1 Confidence Threshold Policy 22
7.1.2 Contradiction Handling 22
7.1.3 Validation Approach 22
7.1.4 Clinician Submission Control 23
7.1.5 Grayscale Rollout and A/B Testing 23
7.1.6 Emergency Rollback and Golden Release 24
7.2 Priority Matrix 25
7.3 Operational Monitoring 26
7.4 Assumptions 26
7.5 External Dependencies 26
8. Resilience & Edge Case Handling 27
8.1 Handwritten or Unclear Clinical Notes 27
8.2 Low-Confidence Model Output 27
8.3 Contradictory Inputs 27
8.4 External API Failure 28

8.5 Unsupported File Type or Local Processing Failure 28
9. Project Management & Team Contributions 29
9.1 Project Timeline 29
9.2 Team Members & Role Assignments 30
9.3 Recommendations & Future Improvements 30
10. References 31

1. Introduction

1.1 Purpose

This System Analysis Documentation (SAD) presents the technical scope, design decisions, and operating assumptions of the Clinical Diagnostics Triage & Workflow Copilot developed by Team Shrooms for UMHackathon 2026. The document serves as the formal reference for implementation, testing, integration, and demonstration of the desktop-based clinical decision support workflow.

1.2 Key Elements Covered

- Architecture: A standalone desktop system composed of a C# WinForms clinical interface, a local Python AI processing engine, direct GLM integration without LangChain, and mock FHIR R4 structured output generation.
- Data Flows: How typed notes, handwritten note images, and multiple biomedical files move through local preprocessing, model inference, triage synthesis, and standardized output preparation.
- Model Process: The end-to-end workflow covering OCR, NLP summarization, modality routing, TensorFlow and Keras inference, and final triage synthesis.
- Error Handling and Safety: How confidence thresholds, contradiction checks, file validation, and API error handling govern automated outcomes and human review routing.
- Role of Reference: This document establishes a shared technical baseline for development, validation, documentation, and final presentation.

1.3 Solution Overview

The Clinical Diagnostics Triage & Workflow Copilot is designed as a desktop-only clinical support system for consolidated review of physician notes and biomedical evidence. Physicians can submit typed notes or handwritten note images and attach a biomedical file within a single session. The C# WinForms application manages intake and result presentation (Microsoft, n.d.), while a local Python processing engine performs GLM-assisted OCR for handwritten-note digitization, context-aware note preprocessing, NLP summarization when note length exceeds the available context window (Xie et al., 2023), modality-specific inference with TensorFlow and Keras (Abadi et al., 2016; Keras Team, n.d.), and final reasoning support through direct GLM calls. The final result is displayed as a clinician-facing triage summary together with a mock FHIR R4 ClinicalImpression representation (Health Level Seven International, 2019). No internal HTTP API is used between the C# and Python components.

2. Background

2.1 Problem Statement

Clinical personnel frequently work with fragmented diagnostic information across separate interfaces, file repositories, and documentation workflows. Physicians may need to interpret ECG signals, review chest imaging, and reconcile handwritten or lengthy narrative notes without a unified support layer. This fragmentation delays triage decisions, increases transcription effort, and raises the likelihood of missed or inconsistent interpretation when time pressure is high.

The problem becomes more severe when unstructured notes and biomedical evidence are not presented in a common workflow. Handwritten notes must be re-read manually, related biomedical files may be opened in separate tools, and the final clinical summary must often be reformatted into documentation standards. A single desktop workflow that consolidates note preparation, file-level inference, and structured output can reduce these process gaps while preserving the need for human review when automation is not sufficiently confident.

2.2 Previous State

Existing diagnostic workflows typically rely on disconnected tools and manual synthesis:

- ECG files are reviewed in dedicated software outside the physician documentation workflow.
- Chest X-ray images are opened separately and interpreted alongside narrative notes rather than within the same decision-support session.
- Handwritten notes remain difficult to search, summarize, and correlate with attached biomedical evidence.
- Physicians manually consolidate observations from multiple inputs and reformat the conclusion into a usable documentation structure.

These limitations create unnecessary cognitive load and reduce process consistency, especially when more than one biomedical file must be considered in a single case.

2.3 Changes in Major Architectural Components

The proposed system replaces fragmented review steps with a controlled desktop workflow that integrates local note handling, model execution, and structured result generation.

Component	Previous State	New State (This System)
Input Capture	Notes and files are handled across separate tools with inconsistent session structure.	A single desktop session accepts typed notes, handwritten note images, and multiple biomedical files.
Clinical Note Preparation	Lengthy or handwritten notes require manual reading and manual summary construction.	GLM-assisted OCR converts handwritten notes into text, and an NLP summarization node condenses notes when the context window is exceeded.
Diagnostic Execution	Each modality is interpreted in isolation with limited workflow coordination.	A local AI engine validates and routes each attached file to the appropriate TensorFlow and Keras model.
System Integration	Information is transferred manually between tools or separate systems.	C# WinForms and Python interact locally through a process bridge and structured JSON payload exchange without an internal HTTP API.
Final Result Formatting	Outputs are manually summarized and reformatted for documentation.	The system returns a clinician-facing triage summary together with a mock FHIR R4 ClinicalImpression output.
Safety Controls	Confidence and contradiction checks depend largely on manual interpretation.	A 90 percent confidence threshold, contradiction detection, validation, and human-review routing govern automated output.

3. Target Stakeholders & Roles

The following table defines the principal stakeholders, their responsibilities in the workflow, and the expected system behavior relevant to each group.

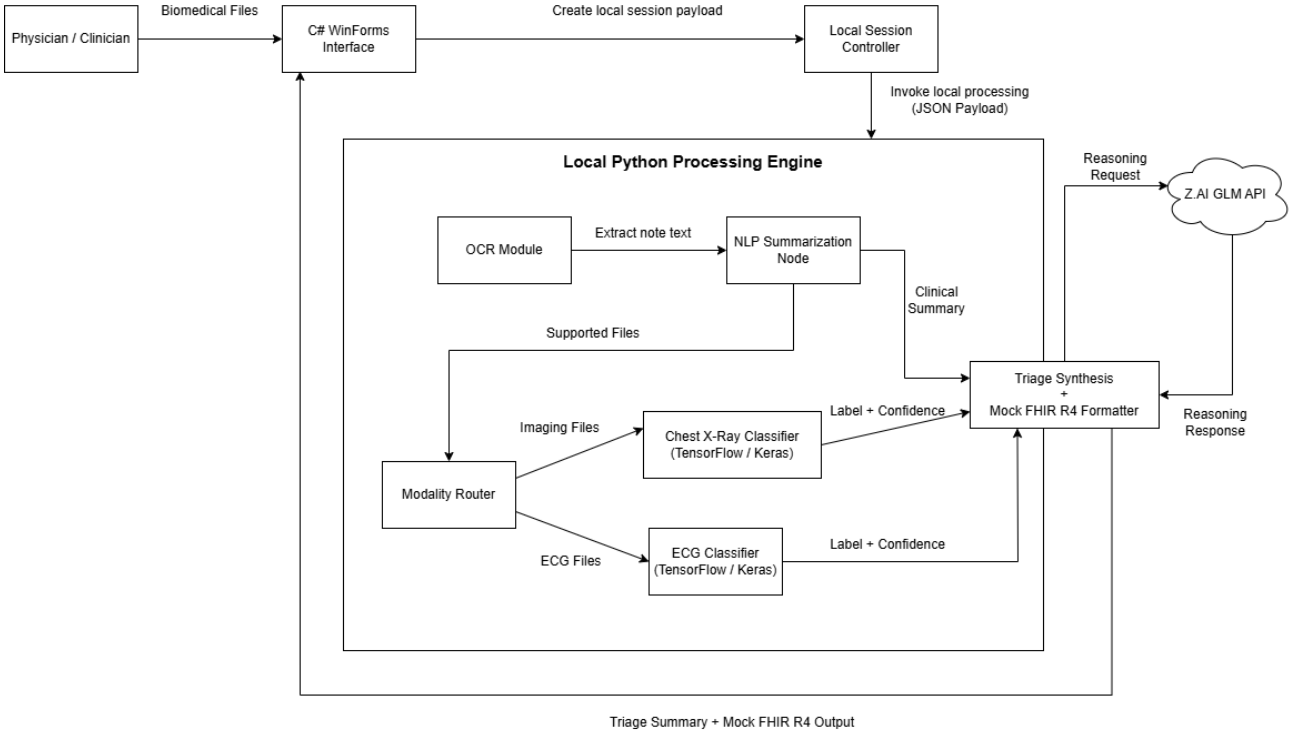
Stakeholder	Role	Expectations
Physician / Clinician	Primary end user who enters notes, optionally uploads a biomedical file, and reviews the generated result.	A fast, consolidated workflow with support for handwritten notes, standardized output, and explicit routing to human review when required.
Nursing / Clinical Support Staff	Assist with note capture, biomedical file acquisition, and operational follow-up for flagged cases.	Clear guidance on why a case requires human review and which input requires attention.
Hospital Administrator / Documentation Reviewer	Reviews the structure and completeness of the generated output for documentation consistency.	A mock FHIR R4-aligned result that is readable, traceable, and suitable for demonstration of standards awareness.
Development Team	Builds and maintains the desktop interface, local Python engine, and integration contract.	Clear module boundaries, a maintainable local deployment model, and no internal HTTP API dependency.
QA / Validation Team	Verifies preprocessing behavior, inference outputs, and triage rules across normal and edge-case scenarios.	Well-defined labels, threshold policy, file-validation rules, and predictable handling of handwritten or ambiguous inputs.
Judges / Demonstration Reviewers	Evaluate system coherence, technical feasibility, and documentation quality during review.	A professional, standards-aware workflow supported by clear architecture and well-placed diagrams.

4. System Architecture & Design

4.1 High-Level Architecture Overview

The system is designed as a single desktop deployment running on a Windows workstation. The user-facing layer is a C# WinForms application responsible for clinical note entry, file attachment, session control, and presentation of final results. The analytical layer is a local Python processing engine invoked by the desktop application through a local process bridge and structured JSON payload exchange.

Because the solution is desktop-only, no internal HTTP API or standalone backend server is introduced between the presentation layer and the AI processing layer. External API traffic is limited to direct outbound calls required for GLM-assisted summarization or reasoning. This keeps the architecture aligned with the intended deployment model while preserving support for advanced language-based processing where needed.



4.2 Clinical Reasoning and Local Processing Flow

The local Python engine coordinates note preprocessing, handwritten note handling, context-window management, file-level inference, and final result preparation. Direct GLM integration is used without LangChain, and the orchestration logic remains within the desktop workflow.

4.2.1 Prompt Construction, Context Management, and Summarization

- Typed clinical notes are received directly from the WinForms interface as raw text.
- Handwritten clinical notes submitted as images are first processed through OCR to obtain machine-readable text.
- If the combined note content exceeds the configured context window, a dedicated NLP AI node summarizes the note while preserving symptoms, duration, and clinically relevant observations.
- The summarized or full note is packaged with validated file metadata and model outputs before final reasoning support is performed.
- All GLM interactions are executed through direct API calls from the Python module, with no LangChain dependency in the processing path.

4.2.2 Multi-File Processing and Modality Routing

- Physicians may attach more than one biomedical file in a single diagnostic session.
- Each file is validated for type and modality before inference is attempted.
- ECG signal files are routed to the ECG classifier, while chest X-ray images are routed to the imaging classifier.
- Results from all attached files are aggregated into a common session object for final synthesis.
- Automated triage synthesis proceeds only when all contributing model outputs satisfy the confidence threshold.

4.3 Use Case Diagram

Overview

This Use Case Diagram maps the functional interactions between the primary end-user, the internal desktop system processes, and the external AI service. It illustrates the two primary workflows available to the user: submitting a new diagnostic case for triage and reviewing historical system activity.

System Actors

Primary Actor (Physician / Clinician): The human user who interacts with the desktop interface to input clinical data, attach files, and retrieve triage summaries.

Secondary Actor (Z.AI GLM API): The external Large Language Model service accessed via API, responsible for providing reasoning capabilities to synthesize the final triage result.

Core Workflow 1: Submit Case for Triage

This is the primary diagnostic pipeline. The physician initiates the process, which branches into clinical note handling and biomedical file processing before final synthesis.

Handling Clinical Notes:

The base action is Prompt Clinical Notes. This can be fulfilled by two extension use cases: Enter Clinical Notes Text (manual typing) or Attach Clinical Note File. If a file is attached, the system handles it via Upload Document or Upload Image. Crucially, if an image is uploaded, the system mandates the Perform OCR on Handwritten Note use case (<<include>>), which extracts the text to fulfill the Get Clinical Note requirement.

Handling Biomedical Files:

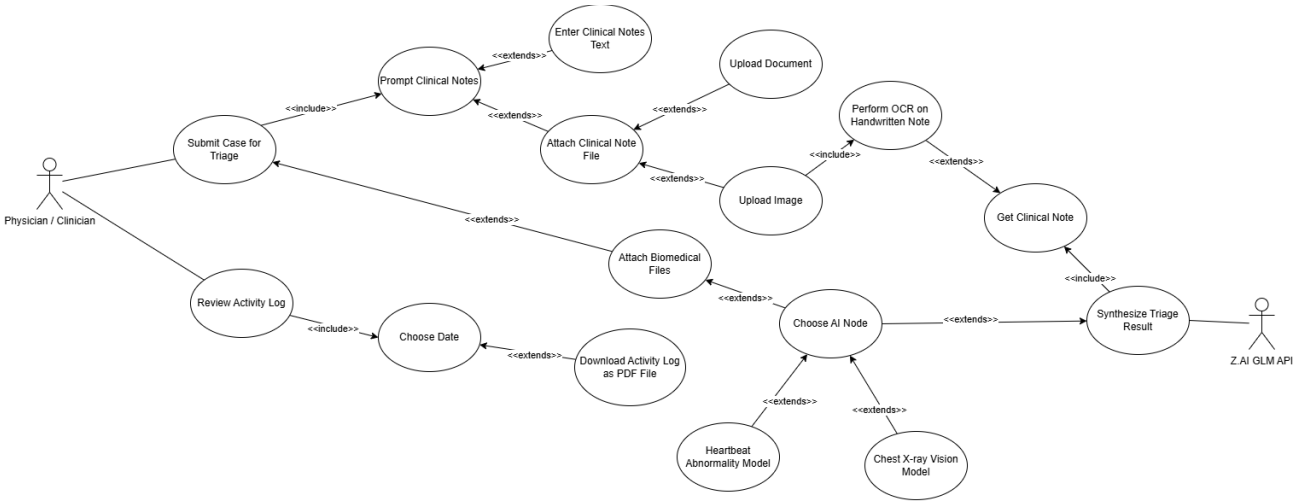
The case submission can be extended by Attach Biomedical Files. When a file is attached, it triggers the Choose AI Node extension, representing the system's routing logic. Depending on the file modality, this extends to executing either the Heartbeat Abnormality Model (for ECGs) or the Chest X-ray Vision Model (for imaging).

Triage Synthesis:

The Synthesize Triage Result use case serves as the convergence point. It takes the prepared text (<<include>> from Get Clinical Note) and the model outputs (<<extends>> from Choose AI Node). This synthesis step interacts directly with the external Z.AI GLM API actor to evaluate the combined evidence and generate the final structured clinical impression.

Core Workflow 2: Review Activity Log

This secondary workflow allows the clinician to audit system actions, AI executions, and routing decisions. The physician initiates the Review Activity Log use case. To view specific logs, this action mandates the Choose Date use case (<<include>>) to filter the historical data. Once a date is selected, the user has the optional extension (<<extends>>) to Download Activity Log as PDF File for external record-keeping or compliance audits.



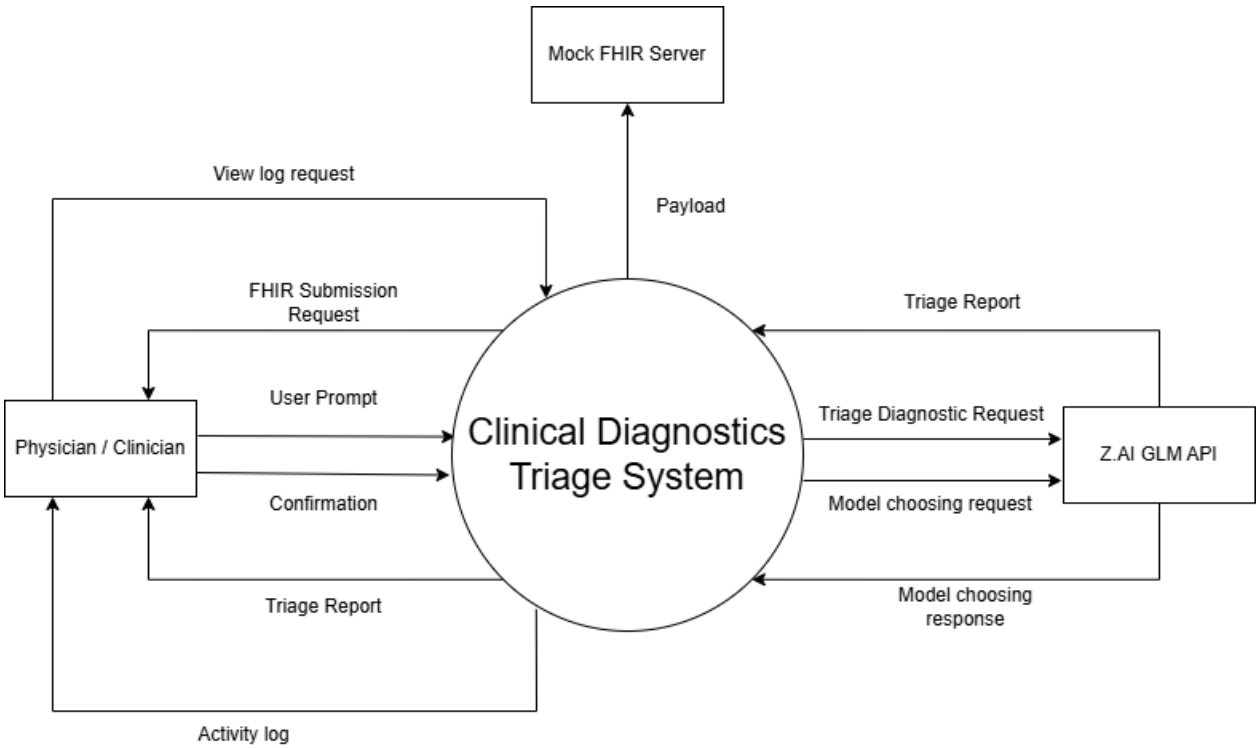
4.4 Technology Stack

Layer	Technology	Purpose
Desktop Presentation	C# WinForms (.NET Framework)	Provides the clinician interface for note entry, multiple file attachment, case submission, and result display.
Local Session Bridge	Local process invocation and JSON payload exchange	Invokes the Python processing engine from the desktop application without relying on an internal HTTP API.
Clinical Note OCR	Z.AI GLM API (ilmu-glm-5.1 with Retrieval Tool)	Converts handwritten clinical note images into machine-readable text by uploading the image to the Z.AI file API and invoking the GLM model with a structured OCR system prompt before downstream analysis.
NLP Summarization Node	Direct GLM integration (ilmu-glm-5.1)	Summarizes lengthy notes when the available context window would otherwise be exceeded.
ECG Classification	TensorFlow and Keras	Implements the five-class ECG inference pipeline for heartbeat interpretation.
Imaging Classification	TensorFlow and Keras	Implements chest X-ray classification for Normal and Pneumonia outcomes.
Structured Output	Mock FHIR R4 DiagnosticReport template	Produces a standardized final output aligned with documentation and interoperability expectations.
External AI Service	Z.AI GLM API	Supports note summarization and final reasoning through direct API calls without LangChain.

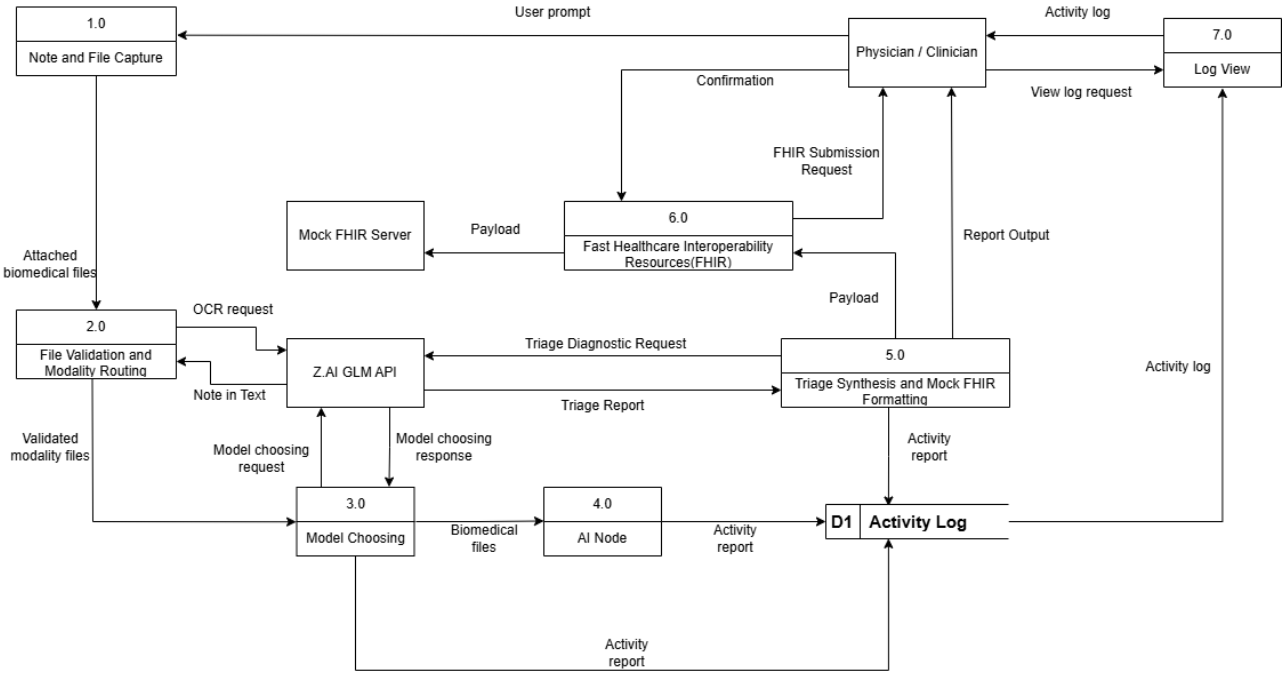
4.5 Data Flow Diagram (DFD)

The data flow is centered on a single desktop workflow. No persistent relational database is required for the core demonstration path; instead, the system uses temporary local session payloads and local logging for controlled execution and traceability.

- External Entity - Physician: Submits typed notes, a handwritten note image, or a single biomedical file.
- Process 1 - Note and File Capture: The WinForms layer packages session inputs and validation metadata.
- Process 2 - Note Digitization and Summarization: OCR converts handwriting to text, and the NLP node summarizes notes when needed.
- Process 3 - File Validation and Modality Routing: The Python engine validates each file and routes it to the appropriate model.
- Process 4 - TensorFlow and Keras Inference: ECG and imaging models produce labels and confidence scores.
- Process 5 - Triage Synthesis and Mock FHIR Formatting: The orchestration layer applies the threshold policy, invokes GLM support when appropriate, and prepares the final structured output.
- External Service - Z.AI GLM API: Provides language-based summarization and reasoning support through direct outbound API calls.
- Output - WinForms Result View: Displays the triage summary, review status, and mock FHIR R4 representation.



Caption: Hierarchical diagram showing the mock ClinicalImpression root, patient reference, clinical summary, triage-priority extension, flagged-for-review extension, finding block, and AI investigation block.

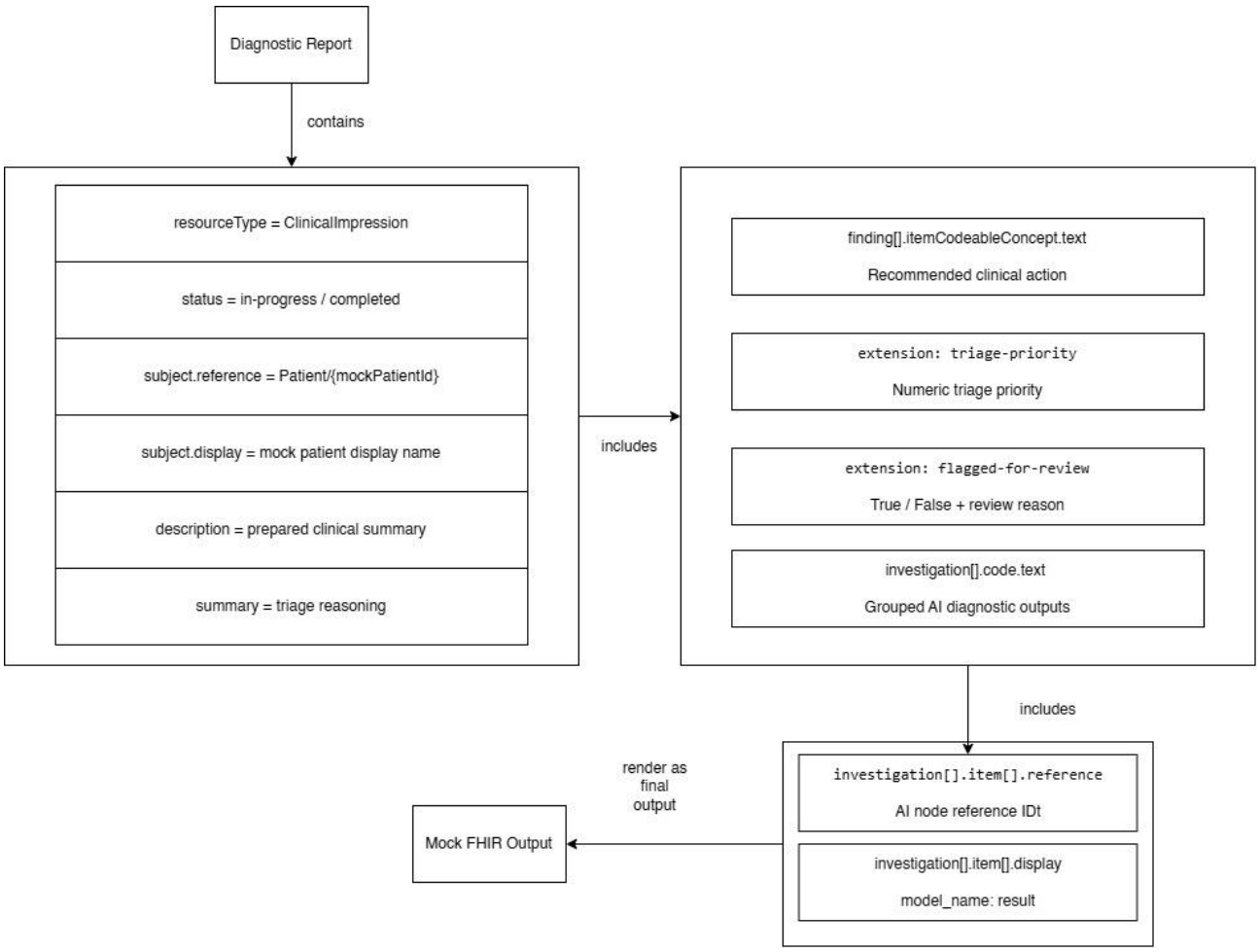


Caption: Level 1 DFD showing note capture, OCR and summarization, modality routing, TensorFlow and Keras inference, safety checks, and mock FHIR R4 output formatting.

4.6 Data Schema - Mock FHIR R4 Output Structure

The desktop workflow does not depend on a conventional relational database for the core case path. Instead, the final output includes a mock FHIR R4 ClinicalImpression-compatible structure that represents the synthesized result in a standardized and demonstrable format aligned with the ClinicalImpression resource model in FHIR R4 (Health Level Seven International, 2019).

FHIR Field	Resource Type	Description
resourceType	ClinicalImpression	Root resource name for the structured output.
status	code	Set to in-progress when the case is flagged for human review, otherwise completed.
subject.reference	Patient	Stores a de-identified mock patient reference constructed from the patient ID.
subject.display	string	Stores the mock patient display name.
description	string	Contains the prepared clinical summary text for the session.
summary	string	Contains the triage reasoning that justifies the recommended action.
finding[]. itemCodeableConcept.text	CodeableConcept	Records the recommended clinical action derived from triage synthesis.
extension: triage-priority	Extension	Captures the numeric triage priority assigned to the case.
extension: flagged-for-review	Extension	Indicates whether the case was flagged for human review due to low confidence, contradiction, or processing failure.
investigation[] .code.text	CodeableConcept	Labels the grouped automated AI diagnostic node outputs. Present only when AI node results exist.
investigation[] .item[].reference	Reference	Stores the reference ID of each contributing AI diagnostic node.
investigation[] .item[].display	string	Displays each AI node's model name and result in model_name: result format.



Caption: Hierarchical diagram showing the mock DiagnosticReport root, patient and practitioner references, observation summary, reviewRequired extension, and presentedForm output block.

5. Diagnostic Models

The system uses two modality-specific classification models implemented with TensorFlow and Keras, supported by a dedicated clinical note preprocessing path for handwriting and long-context management. Model routing is determined by file type, and note preparation is completed before final synthesis is attempted.

5.1 ECG Signal Classification

- Modality: One-dimensional ECG signal files.
- Model Architecture: TensorFlow and Keras sequence classifier for heartbeat pattern recognition.
- Task: Five-class heartbeat classification.
- Classification Labels: "Normal", "Supraventricular Ectopic Beats", "Ventricular Ectopic Beats", "Fusion Beats", and "Unknown".
- Input Handling: One or more ECG files may be attached in a single session, and each file is evaluated independently before aggregation.
- Output: Predicted label and confidence score for each ECG file.
- Decision Rule: Any ECG output below 90 percent confidence routes the case to human review rather than autonomous triage synthesis.

5.2 Medical Imaging Analysis

- Modality: Two-dimensional chest X-ray images.
- Model Architecture: Convolutional neural network implemented with TensorFlow and Keras.
- Task: Binary medical image classification.
- Classification Labels: "Normal" and "Pneumonia".
- Input Handling: Multiple chest imaging files may be attached and analyzed individually within the same session.
- Output: Predicted label and confidence score for each image.
- Decision Rule: Any imaging output below 90 percent confidence routes the case to human review rather than autonomous triage synthesis.

5.3 Clinical Note NLP Processing

- Input Types: Typed clinical notes and handwritten note images submitted as part of the same desktop session.
- Preprocessing: Handwritten note images are uploaded to the Z.AI file API and processed by the GLM model (ilmu-glm-5.1) operating under a structured OCR system prompt, which returns machine-readable text before downstream reasoning proceeds.
- Summarization Node: An NLP AI node is invoked when the note length exceeds the configured context window, consistent with biomedical text summarization practice (Xie et al., 2023).
- Purpose: Preserve clinically relevant symptoms, observations, and temporal details before final synthesis.
- Safety Condition: If note extraction or summarization fails, the system does not generate autonomous triage and instead requests human review.
- Direct Integration: GLM calls are made directly from the Python module without LangChain.

6. Functional Requirements & Scope

6.1 Core Functional Capabilities

Feature	Description
Desktop Clinical Note Intake	Physicians submit typed clinical notes or handwritten note images within a single C# WinForms session. OCR converts handwritten images into machine-readable text before downstream processing.
File Submission	A single biomedical file may be attached per diagnostic session. The file is validated for type and modality before inference is attempted. Multi-file parallel processing is architecturally supported at the Python engine layer and is planned as a desktop interface enhancement.
GLM-Driven Model Routing	The first of two GLM API calls (ilmu-glm-5.1) acts as a routing agent. It receives the physician's notes and attached file metadata and determines which local AI model to invoke Heartbeat_Abnormality_Model for ECG signal files or Chest_XRay_Vision_Model for chest X-ray images returning a structured JSON routing decision.
Local ECG Signal Classification	ECG files are routed to a TensorFlow and Keras five-class heartbeat classifier. The model returns a predicted label and confidence score for each file submitted.
Local Chest Imaging Classification	Chest X-ray images are routed to a convolutional neural network that produces a Normal or Pneumonia label and confidence score for each image submitted.
GLM-Driven Triage Synthesis and Structured Output	A second GLM API call synthesizes physician notes with all AI node results. When all contributing outputs meet or exceed 90 percent confidence and no contradiction is detected, a structured triage JSON is generated. flagged_for_human_review is set to true otherwise. The result is displayed in the WinForms chat panel and formatted as a mock FHIR R4 ClinicalImpression output by FHIR_R4_Handling.py.
Session Activity Logging	Every API call, OCR execution, and AI node execution is written to a date-stamped TSV file under Activity_Log/ via activity_logger(), recording a unique activity ID composed of a Unix timestamp, line count, and source identifier, along with an event type label and the full response or error payload.

6.2 Non-Functional Requirements

NFR	Requirement	Implementation Approach
Reliability	Every session must produce either a structured triage result or an explicit human-review outcome without silent failure.	Local validation, structured exception handling on direct API calls, safe fallback messages, and controlled result states are applied.
Usability	A clinician should be able to complete the workflow quickly within a single desktop interface.	The WinForms layout centralizes note entry, file upload, submission, and result review in one session flow.
Maintainability	The presentation logic and AI processing logic must remain independently maintainable.	C# and Python exchange data through a stable local JSON contract and a controlled invocation bridge.
Performance	Routine preprocessing and inference should complete within practical demonstration latency on a workstation environment.	Local inference is used wherever possible, and the NLP summarization node is activated only when needed by context size.
Interoperability	The final output should reflect a standards-aware structure suitable for documentation demonstration.	A dedicated formatter maps the result into a mock FHIR R4 ClinicalImpression representation.
Safety	Autonomous triage synthesis is permitted only when all contributing evidence satisfies the confidence policy.	A 90 percent confidence gate, contradiction handling, OCR quality checks, and human-review routing are enforced.
Token Latency	Each of the two GLM API calls made per session must complete within a practical timeframe to keep the demonstration responsive. The WinForms UI displays a live "Analyzing..." pulse animation at 400 ms intervals while the Python backend is running.	zai.ZAIError.TokenLimitError is explicitly caught on both the model-choosing and final-analysis calls and surfaced as a controlled error in the WinForms chat panel. A hard API timeout parameter on the GLM calls is a planned addition before the demonstration build.

Cost Efficiency	The token volume consumed across both GLM calls per session must remain predictable. Both system prompts are fixed-length strings defined at build time, establishing a stable baseline token cost per session.	Binary biomedical files (ECG, imaging) are uploaded to the Z.AI file API and referenced by ID rather than inlined into the prompt, keeping binary content out of the context window. Text-based file content is injected via file_as_prompt and is bounded by the provider's token ceiling, which triggers TokenLimitError handling. NLP summarization to condense oversized notes before injection is a planned addition

6.3 Out of Scope / Future Enhancements

The following items are not part of the present implementation scope but are future enhancements.

- Direct write-back to a live production EHR environment.
- Mobile or web-based client interfaces.
- Continuous streaming intake for real-time bedside monitoring devices.
- Full DICOM-native imaging preprocessing and metadata preservation.
- Long-term ML model confidence trend tracking across deployment periods.

7. Monitoring, Evaluation, Assumptions & Dependencies

7.1 Technical Evaluation Strategy

7.1.1 Confidence Threshold Policy

Each model output includes a confidence value within the range of 0.0 to 1.0. For sessions containing more than one biomedical file, all contributing outputs are considered together. Automated triage synthesis is permitted only when every contributing model output reaches or exceeds 0.90 confidence.

- Confidence greater than or equal to 0.90: Eligible for automated triage synthesis.
- Confidence below 0.90: Human review is required, and autonomous synthesis is withheld.
- If multiple biomedical files are attached, a single low-confidence contributing result is sufficient to route the entire session to human review.

7.1.2 Contradiction Handling

- If the prepared clinical note materially conflicts with the model outputs, the system withholds autonomous triage and routes the session to human review.
- The mock FHIR output records the review condition through the reviewRequired extension and an accompanying review reason.

7.1.3 Validation Approach

- ECG and imaging models are evaluated on held-out validation data before inclusion in the desktop workflow.
- Evaluation should consider label-level precision, recall, and class consistency for both models.
- Handwritten-note testing should cover both successful OCR extraction and unreadable-note fallback scenarios.

7.1.4 Clinician Submission Control

FHIR_R4_Handling.py constructs the mock FHIR R4 ClinicalImpression payload from the triage JSON produced by the Python engine and the patient metadata passed in as arguments. The current implementation calls mock_push_to_fhir_server(), which prints the final JSON payload to stdout for demonstration purposes. Integration of this script into the WinForms workflow including display of the FHIR payload in a readable UI panel and an explicit clinician confirmation step before submission is a planned addition before the demonstration build.

7.1.5 Grayscale Rollout and A/B Testing

Because this system operates as a desktop deployment rather than a continuously served web application, the rollout strategy is applied at the model package and configuration level rather than at the infrastructure traffic level.

- **Grayscale Rollout.** When a new ECG or imaging model version is prepared, it is introduced to a controlled subset of validation sessions before replacing the production model package. The rollout follows a staged promotion sequence: the new model package is first evaluated on 10 percent of held-out validation cases. If label-level precision, recall, and confidence distribution remain consistent with the baseline over that sample, the package is promoted to 50 percent of validation cases. Full promotion to the active desktop deployment occurs only after the expanded sample confirms no degradation. If any stage surfaces a regression, the candidate package is withheld and the current stable version remains active.
- **A/B Testing.** Two configuration variants are compared to determine which produces safer and more consistent triage routing without increasing unnecessary human-review volume. Variant A applies the current 90 percent confidence threshold uniformly across all contributing model outputs. Variant B applies an 85 percent threshold for ECG outputs and retains 90 percent for imaging outputs, reflecting the higher label stability observed in the ECG classifier during held-out validation. Each variant is evaluated across the same held-out case set. The variant that achieves a lower unnecessary-review rate without producing any high-confidence misclassification is adopted as the active configuration for the demonstration build.

7.1.6 Emergency Rollback and Golden Release

A Golden Release is defined as the last model package and configuration set that passed full staged validation and produced no safety-critical misclassification across the held-out validation cases. The Golden Release is retained as a versioned local archive on the deployment workstation and is never overwritten by a candidate package that has not completed the grayscale promotion sequence.

Trigger Conditions. Emergency rollback is initiated when any of the following conditions are detected during post-deployment validation or live demonstration:

- A newly promoted model package produces a high-confidence output (at or above 0.90) that contradicts the ground-truth label on a known reference case.
- The human-review routing rate across a representative validation sample rises by more than 15 percentage points relative to the Golden Release baseline, indicating a confidence regression.
- A local runtime failure or model loading error prevents the Python processing engine from completing inference for any supported modality.

Rollback Procedure. When a trigger condition is confirmed, the active model package and configuration file are replaced with the archived Golden Release versions. No retraining or re-promotion is required for rollback. The desktop application is relaunched to load the restored package, and a brief validation pass over the reference case set is run to confirm stable behaviour before the session resumes.

Priority Classification. Rollback trigger conditions are treated as P1 events under the Priority Matrix defined in Section 7.2. The clinician-facing interface is not used for demonstration purposes until the Golden Release has been confirmed as restored and validated.

7.2 Priority Matrix

The following priority matrix defines the resulting action path after evidence aggregation and confidence evaluation.

Priority	Trigger Condition	System Action	Final Output
P1 - Immediate Attention	A clinically severe finding is identified, all contributing outputs are at or above 0.90 confidence, and the synthesized note context supports escalation.	The system generates an automated triage synthesis with urgent recommended action.	Conclusion: "Immediate Attention Required". triage_priority: 1
P2 - Priority Clinical Review	A non-normal finding is identified, all contributing outputs are at or above 0.90 confidence, and no contradiction is detected.	The system generates an automated triage synthesis with priority follow-up guidance.	Conclusion: "Priority Clinical Review Required". triage_priority: 2
P3 - Routine Clinical Review	Findings are lower acuity or normal, all contributing outputs are at or above 0.90 confidence, and note content is consistent..	The system generates an automated triage summary for routine clinician review.	Conclusion: "Routine Clinical Review". triage_priority: 3
P4 - Non-Urgent	Findings indicate no clinically actionable concern, all contributing outputs are at or above 0.90 confidence, and note content is consistent.	The system generates an automated triage summary marking the case as non-urgent.	Conclusion: "Non-Urgent". triage_priority: 4
P5 - Human Review Required	Any contributing model output is below 0.90 confidence, OCR quality is insufficient, a contradiction is detected, or a required API call fails.	Autonomous synthesis is withheld, the case is flagged for human review, and the clinician is notified.	Conclusion: "Human Review Required". triage_priority: 5

7.3 Operational Monitoring

- Local session logging records session identifiers, file counts, detected modalities, and final processing state.
- API call failures are caught by the Python engine and surfaced as a "System Error" message in the WinForms chat panel. Both zai.ZAIError.TokenLimitError and general exceptions are written to the Activity_Log TSV via activity_logger() before the RuntimeError is raised.
- Result traceability records whether autonomous synthesis was permitted or whether human review was required for the session.

7.4 Assumptions

- The system runs on a Windows workstation capable of executing a C# WinForms application and a local Python runtime.
- The Python executable path is currently hardcoded to the development workstation at C:\Users\weisi\AppData\Local\Programs\Python\Python312\python.exe. A configurable path is required before demonstration on any other machine.
- The current C# WinForms implementation supports attachment of one biomedical file per session. Multi-file processing is architecturally supported at the Python engine layer and is a planned desktop enhancement.
- The currently supported biomedical inputs are ECG signal files and chest X-ray images.
- A stable network connection is available when outbound GLM API calls are required.
- Mock patient and practitioner identifiers are acceptable for demonstration-oriented FHIR output generation.

7.5 External Dependencies

Dependency	Provider	Purpose & Risk Mitigation
Z.AI GLM API (ilmu-glm-5.1)	Z.AI	Supports GLM-assisted OCR, model routing, note summarization, and final triage reasoning. TokenLimitError and general exceptions are caught, logged to Activity_Log TSV, and surfaced as errors in the WinForms interface.
TensorFlow and Keras	Open Source	Provide the local model runtime for ECG and chest imaging inference on the desktop environment.
Mock FHIR R4 Output Template	Internal Project Asset	Provides a standardized demonstration output aligned with DiagnosticReport structure.

8. Resilience & Edge Case Handling

The desktop workflow is designed to avoid unsafe automation when inputs are incomplete, ambiguous, or technically unreliable. The system therefore routes unresolved situations into explicit human-review states rather than generating unsupported autonomous conclusions.

8.1 Handwritten or Unclear Clinical Notes

Scenario: A handwritten clinical note is submitted as an image file and requires conversion before synthesis can proceed.

- OCR attempts to convert the handwritten note into machine-readable text.
- If extraction succeeds, the resulting text continues through summarization and downstream processing.
- If handwriting is unclear or the extracted content is incomplete, the session is routed to human review and the issue is recorded in the structured output.

8.2 Low-Confidence Model Output

Scenario: One or more biomedical files produce a model confidence score below the required threshold.

- The affected result is marked as insufficient for autonomous triage synthesis.
- The entire session is routed to human review because every contributing result must satisfy the threshold policy.
- The desktop interface displays the relevant file-specific confidence value and the reason for the review requirement.

8.3 Contradictory Inputs

Scenario: The prepared clinical note materially conflicts with the labels produced by the biomedical models.

- The contradiction is treated as a safety-critical uncertainty condition.
- Autonomous triage is withheld, and the session is routed to human review.
- The review reason is captured in the structured output for traceability.

8.4 External API Failure

Scenario: A direct GLM API call returns an error response or fails during execution.

- If the API call fails, the Python engine raises a RuntimeError which the C# process bridge catches and displays as a controlled error in the WinForms chat panel. No partial synthesis is shown.
- The failure event is written to the Activity_Log TSV via activity_logger() with the relevant event type (Model_Choosing_API_Error, Model_Choosing_Token_Limit_Error, Final_Analysis_API_Error, or Final_Analysis_Token_Limit_Error) before the RuntimeError is raised.

8.5 Unsupported File Type or Local Processing Failure

Scenario: An attached file is unsupported or a local processing exception prevents successful inference.

- Input files are validated before inference is attempted.
- Unsupported file types or local runtime failures produce explicit status messages rather than silent failure.
- If the affected file is required for the case, the session is routed to human review and remains available for correction or manual follow-up.

9. Project Management & Team Contributions

9.1 Project Timeline

The following timeline outlines the 10-day preliminary development period for UMHackathon 2026.

Phase	Duration	Key Deliverables
Phase 1 - Planning and Analysis	Days 1 to 2	Finalize the desktop-only architecture, define the local data contract, and confirm the documentation structure.
Phase 2 - Model and OCR Preparation	Days 2 to 4	Prepare ECG and imaging models, define label mappings, and establish handwritten note preprocessing behavior.
Phase 3 - Desktop Application Development	Days 2 to 5	Implement the WinForms intake interface, file upload flow, validation states, and result panel.
Phase 4 - AI Processing Integration	Days 4 to 6	Implement the local C# to Python bridge, direct GLM calls, summarization logic, and mock FHIR output formatting.
Phase 5 - Testing and Validation	Days 6 to 8	Validate threshold routing, handwritten note handling, file attachment, and API failure fallback behavior.
Phase 6 - Documentation and Demonstration Preparation	Days 8 to 10	Finalize the SAD document, diagram planning notes, and end-to-end demonstration narrative.

9.2 Team Members & Role Assignments

Name	Primary Role	Responsibilities
Daniel Chan Zit Fung	Backend Lead, Lead ML Engineer, AI Integration Lead	Architects the local Python processing engine, leads ECG and imaging model work in TensorFlow and Keras, and owns direct GLM integration, summarization logic, and final synthesis flow.
Voon Wei Siang	Frontend Lead, Desktop Integration	Leads development of the C# WinForms interface and implements note capture, file upload, and local invocation of the Python engine.
Ho Kar Seng	Frontend, UX, Structured Output Presentation	Contributes to result rendering, review alerts, and presentation of the final triage summary and mock FHIR R4 output.
Nicholas Ong Yean Bing	Backend, OCR, Validation	Implements input validation, handwritten note handling, API error handling, and structured output support while contributing to validation scenarios.

9.3 Recommendations & Future Improvements

- Introduce long-term ML model confidence trend tracking across deployment periods to support operational analytics and model governance.
- Extend the imaging path to full DICOM-native preprocessing and metadata preservation.
- Add direct certified hospital FHIR integration beyond the current mock FHIR R4 output model.

10. References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2016). TensorFlow: Large-scale machine learning on heterogeneous distributed systems. arXiv. https://doi.org/10.48550/arXiv.1603.04467

Health Level Seven International. (2019). ClinicalImpression - FHIR v4.0.1. HL7 FHIR. https://hl7.org/fhir/R4/clinicalimpression.html

Keras Team. (n.d.). Keras 3: Deep learning for humans. Keras. https://keras.io/keras_3/

Microsoft. (n.d.). What is Windows Forms. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/

Xie, Q., Luo, Z., Wang, B., & Ananiadou, S. (2023). A survey for biomedical text summarization: From pre-trained to large language models. arXiv. https://doi.org/10.48550/arXiv.2304.08763

End of System Analysis Documentation

Team Shrooms | UMHackathon 2026 | Clinical Diagnostics Triage & Workflow Copilot